

Machine Learning — Complete Beginner's Guide

Covering: Linear Regression → Logistic Regression → Regularization → Feature Engineering

SESSION 1: LINEAR REGRESSION

The Big Picture First

Machine learning problems fall into two buckets:

Type	What it predicts	Example
Regression	A number (continuous)	Marks, salary, temperature
Classification	A category	Placed/Not Placed, Spam/Not Spam

Linear Regression is the simplest tool for the first bucket — predicting a number from other numbers.

The core idea: if you know a student studied for x hours, can you predict their marks y ? If more hours generally means more marks, you can draw a straight line through the data points and use that line to predict.

The Model (One Feature)

$$\hat{y} = \beta_0 + \beta_1 x$$

$\hat{y}$ (y-hat) = the **predicted** value

x = input feature (e.g., hours studied)

β_0 (beta-zero) = **intercept** — the predicted value when $x = 0$

β_1 = **slope** — how much y changes per unit increase in x

Worked example from the slides: $\text{Marks} = 2.2 + 0.6 \times \text{Hours}$

Intercept (2.2): if a student studies 0 hours, predicted marks ≈ 2.2

Slope (0.6): every extra hour of study adds 0.6 marks

For 6 hours studied: $\hat{y} = 2.2 + 0.6(6) = \mathbf{5.8 \text{ marks}}$

Key insight: coefficients aren't just numbers — they tell a story about the relationship.

Multiple Linear Regression (More Than One Feature)

Real predictions usually depend on more than one thing. If marks depend on hours studied AND practice tests AND attendance:

$$\hat{y} = \beta_0 + \beta_1 x_1 + \beta_2 x_2 + \cdots + \beta_p x_p$$

Each coefficient β_i tells you: "if this one feature goes up by 1 unit, and everything else stays fixed, the prediction changes by β_i ."

Geometry (visualize it):

1 feature → a **line** in 2D

2 features → a **flat plane** in 3D

3+ features → a **hyperplane** (can't visualize, but same idea — a flat surface)

Residuals — How Wrong Was the Prediction?

$$e_i = y_i - \hat{y}_i$$

This is the **residual** (error) for one data point:

Actual value minus predicted value

Positive residual → your prediction was too low

Negative residual → your prediction was too high

Smaller |residual| → better prediction

The Cost Function: Mean Squared Error (MSE)

We need one single number that tells us "how bad is this line overall?" That's the cost function.

$$MSE = \frac{1}{n} \sum_{i=1}^n (y_i - \hat{y}_i)^2$$

Why square the errors instead of just adding them?

Positive and negative errors don't cancel each other out

Big errors get punished more heavily (an error of 10 becomes 100, way worse than an error of 2 becoming 4)

It's mathematically convenient for optimization (smooth curve, easy to take derivatives)

Training = finding the β values that make MSE as small as possible.

Worked Example (from slides)

x (hours)	y (actual)	$\hat{y}$ (predicted)	$e = y - \hat{y}$	e^2
1	2	2.8	-0.8	0.64
2	4	3.4	0.6	0.36
3	5	4.0	1.0	1.00
4	4	4.6	-0.6	0.36
5	5	5.2	-0.2	0.04

Sum of $e^2 = 2.4 \rightarrow \text{MSE} = 2.4/5 = 0.48$

Other Evaluation Measures

Measure	Formula	What it tells you
MSE	avg of squared errors	Overall error, punishes big mistakes
RMSE	$\sqrt{\text{MSE}}$	Same units as your target (easier to interpret)
MAE	avg of errors	Average error size, treats all errors equally
R^2	$1 - (\text{error variance} / \text{total variance})$	% of variation in y explained by the model (closer to 1 = better)
Adjusted R^2	R^2 adjusted for # of features	Penalizes adding useless features

How Does the Model "Learn" the Best β Values?

Two competing methods, same goal: minimize MSE.

Method 1: Normal Equation (Direct Math Solution)

$$\hat{\beta} = (X^T X)^{-1} X^T y$$

Think of this as solving the problem in **one shot** using matrix algebra — no trial and error.

X = matrix of your input features (with a column of 1s added for the intercept)

y = vector of actual target values

$\hat{\beta}$ = the coefficients you're solving for

Pros: No learning rate to tune, exact answer, done in one step. **Cons:** Gets expensive/slow with lots of features; breaks down if features are exact multiples of each other (matrix isn't invertible).

Method 2: Gradient Descent (Iterative, Trial-and-Improve)

Imagine you're standing on a hilly cost-function surface in the fog, and you want to reach the bottom (lowest error). You can't see the whole landscape, but you can feel which direction is downhill under your feet — so you take a step that way, then check again, and repeat.

The steps:

Start with random β values

Calculate predictions and error (MSE)

Calculate the **gradient** — the direction of steepest INCREASE in error

Step in the OPPOSITE direction (steepest decrease) — that's why we subtract

Repeat until error stops shrinking meaningfully

$$\beta \leftarrow \beta - \alpha \nabla J(\beta)$$

α (alpha) = **learning rate** — how big a step you take each time

$\nabla J(\beta)$ = gradient — tells you which way is "uphill" in error

Learning rate matters a lot:

Learning rate	What happens
Too small	Learns painfully slowly
Just right	Smooth, steady improvement
Too large	Overshoots the minimum, bounces around, may never converge

Normal Equation vs Gradient Descent — When to use which?

Aspect	Normal Equation	Gradient Descent
Method	One-step direct solve	Iterative, step-by-step
Learning rate needed?	No	Yes
Small # of features	Great	Great
Huge # of features	Slow/expensive	Scales much better

SESSION 2: LOGISTIC REGRESSION

Wait — Why Not Just Use Linear Regression for Yes/No Questions?

Say you want to predict "Will this student be Placed (1) or Not Placed (0)?"

If you use plain linear regression: $\hat{y} = w^T x + b$, the output can be **any real number**: -0.4, 1.7, 5.2 — none of these make sense as a "probability of being placed." Probabilities MUST be between 0 and 1.

Fix: Keep computing a linear score, but squash it into a valid probability using a special function.

The Sigmoid Function — The Squashing Trick

$$\sigma(z) = \frac{1}{1 + e^{-z}}$$

This function takes ANY real number ($-\infty$ to $+\infty$) and squishes it into the range (0, 1). It's shaped like a stretched "S."

Key properties:

$\sigma(0) = 0.5$ (right in the middle — total uncertainty)

Large positive $z \rightarrow \sigma(z)$ approaches 1 (very confident "yes")

Large negative $z \rightarrow \sigma(z)$ approaches 0 (very confident "no")

Changes fastest near $z = 0$ (that's the "decision zone")

z	$\sigma(z)$
-4	0.018
-1	0.269
0	0.500
1	0.731
4	0.982

The Full Logistic Regression Pipeline

Linear score: $z = w_1 x_1 + w_2 x_2 + \dots + w_p x_p + b$ (exactly like linear regression!)

Sigmoid: $\hat{p} = \sigma(z) \rightarrow$ converts score into probability

Threshold: if $\hat{p} \geq 0.5 \rightarrow$ predict class 1; else predict class 0

Worked Example

Given CGPA, internships, aptitude score $\rightarrow$ linear score $z = 1.6$ $\sigma(1.6) \approx 0.832 \rightarrow$ since $0.832 > 0.5 \rightarrow$ **Prediction: PLACED**

Important: "The prediction path uses the trained model; it does not retrain the model." Training happens once (using historical data); afterward you just plug in new students' data.

Why Not Use MSE Here? (Binary Cross-Entropy Loss)

MSE doesn't work well for classification because it doesn't punish "confident wrong answers" harshly enough, and its math doesn't play nicely with the sigmoid during training. Instead we use **Binary Cross-Entropy (log loss)**:

$$L(y, \hat{p}) = -[y \log(\hat{p}) + (1 - y) \log(1 - \hat{p})]$$

In plain English:

If the true label is 1: Loss = $-\log(\hat{p}) \rightarrow$ the more confident and correct you are ($\hat{p}$ near 1), the smaller the loss

If the true label is 0: Loss = $-\log(1 - \hat{p}) \rightarrow$ the more confident and correct ($\hat{p}$ near 0), the smaller the loss

Why this matters — confident wrong predictions get PUNISHED hard:

$\hat{p}$ (when $y=1$)	Loss
0.99 (very confident, correct)	0.010 (tiny)
0.70	0.357
0.50 (unsure)	0.693
0.10 (confidently wrong!)	2.303 (huge!)
0.01 (extremely confidently wrong)	4.605 (massive!)

This teaches the model: "don't be confidently wrong — that's the worst possible outcome."

How Logistic Regression Learns

Same overall idea as linear regression's gradient descent, just swap MSE for cross-entropy:

$$w \leftarrow w - \alpha \frac{\partial J}{\partial w} \quad b \leftarrow b - \alpha \frac{\partial J}{\partial b}$$

Decision Boundary

The **decision boundary** is the line (or plane) where the model is exactly 50/50 undecided — where $\hat{p} = 0.5$.

Since $\sigma(0) = 0.5$, the boundary is simply:

$$w^T x + b = 0$$

Points on one side $\rightarrow$ predicted class 1 (Placed). Points on the other side $\rightarrow$ class 0 (Not Placed). Points near the boundary are the "unsure" cases.

Geometry (same pattern as before):

1 feature $\rightarrow$ boundary is a point

2 features $\rightarrow$ boundary is a line

3 features $\rightarrow$ boundary is a plane

More $\rightarrow$ hyperplane

Note: feature scaling matters here too! If CGPA (0-10) and Aptitude Score (0-100) aren't scaled similarly, the score z gets dominated by whichever feature has bigger raw numbers.

Worked Example (Pass/Fail by study hours)

Given $w=1$, $b=-5$. Predict pass/fail for $x=7$ hours.

$$z = wx + b = 1(7) - 5 = 2$$

$$\hat{p} = \sigma(2) = 1/(1+e^{-2}) \approx 0.881$$

$0.881 > 0.5 \rightarrow$ **PASS**

Boundary: $x - 5 = 0 \rightarrow x = 5$ (the model flips its prediction exactly at 5 hours)

Beyond Two Classes: Softmax (Multiclass)

What if there are 3+ categories, like "Low / Medium / High" performance? A single sigmoid isn't enough because we need probabilities across ALL classes that **sum to exactly 1**.

Solution: Softmax. Compute one linear score per class, then convert to probabilities:

$$P(y = k \mid x) = \frac{e^{z_k}}{\sum_{j=1}^K e^{z_j}}$$

In plain English: take the exponential of each class's score, then divide by the sum of all exponentials. This guarantees every probability is between 0–1 and they all add up to 1.

Worked Example

Student studied 5 hours, 75% attendance. Trained weights give scores:

Class	Score z	Softmax Probability
-------	-----------	---------------------

Low	-0.25	0.126 (12.6%)
-----	-------	---------------

Medium	0.875	0.388 (38.8%)
--------	-------	---------------

High	1.10	0.486 (48.6%)
------	------	---------------

Highest probability wins → **Prediction: HIGH performance**

SESSION 3: REGULARIZATION

The Core Problem: Overfitting

Imagine two students preparing for an exam:

Student 1 (learns concepts): understands underlying ideas, does fine on both familiar AND new questions → **generalizes well**

Student 2 (memorizes exact questions): aces practice questions but can't answer anything slightly different → **overfits**

A machine learning model can do the exact same thing — memorize the *noise* in the training data instead of learning the real pattern.

Signs of overfitting:

Very low error on training data

High error on new/test data

The model has basically "memorized" quirks specific to the training set rather than learning something that transfers

What Is Regularization?

The fix: add a penalty to the cost function that discourages the model from becoming overly complex (having huge coefficients).

$$\text{Total Loss} = \text{Original Loss (MSE)} + \text{Regularization Penalty}$$

This forces the model to:

Use smaller coefficient values

Not lean too heavily on any single feature

Learn simpler, more generalizable relationships

Important: regularization is NOT about making training error the smallest possible — it's about making the model perform well on NEW, unseen data.

The Regularization Strength: λ (lambda)

$$J(w) = MSE + \lambda \times \text{Penalty}$$

λ controls HOW MUCH we punish complexity:

λ value	Effect	Risk
$\lambda = 0$	No regularization at all	Overfitting
Small λ	Weak penalty	Still overfit-prone
Moderate λ	Balanced	Sweet spot
Large λ	Strong shrinkage	—
Very large λ	Coefficients crushed toward 0	Underfitting

Three Flavors of Regularization

1. L2 / Ridge Regression — Penalize Squared Coefficients

$$J_{\text{Ridge}} = \frac{1}{n} \sum (y_i - \hat{y}_i)^2 + \lambda \sum w_j^2$$

Shrinks all coefficients toward small values

Rarely sets any coefficient exactly to 0 — keeps every feature "a little bit active"

Great when correlated features are present and most features genuinely matter

Example result: $[4.8, 3.7, 0.9, 0.4] \rightarrow [3.2, 2.6, 0.7, 0.3]$ (everything got smaller, nothing became zero)

2. L1 / Lasso Regression — Penalize Absolute Coefficients

$$J_{Lasso} = \frac{1}{n} \sum (y_i - \hat{y}_i)^2 + \lambda \sum |w_j|$$

Can force some coefficients to become **exactly zero**

This effectively removes irrelevant features from the model — automatic feature selection!

Great when you suspect many features are useless

Example result: $[4.8, 3.7, 0.9, 0.4] \rightarrow [3.5, 2.8, 0, 0]$ (two features got completely eliminated)

3. Elastic Net — Best of Both Worlds

$$J_{ElasticNet} = \frac{1}{n} \sum (y_i - \hat{y}_i)^2 + \lambda \left(\alpha \sum |w_j| + (1 - \alpha) \sum w_j^2 \right)$$

α controls the mix between L1 and L2:

$\alpha = 1 \rightarrow$ pure Lasso

$\alpha = 0 \rightarrow$ pure Ridge

$0 < \alpha < 1 \rightarrow$ blend of both

Useful when features are BOTH correlated AND some are irrelevant.

Comparison Table

Aspect	Ridge	Lasso	Elastic Net
Penalty type	L2 (squared)	L1 (absolute)	L1 + L2
Sets coefficients to zero?	Usually no	Yes	Can
Does feature selection?	No	Yes	Yes
Correlated features	Handles well	May arbitrarily pick one	Handles better
Best when	Most features matter	Many irrelevant features	Both correlated + irrelevant features

Sparsity

A **sparse** model is one where many coefficients are exactly zero — meaning those features simply don't contribute to predictions at all.

Example: $w = [2.4, 0, 1.7, 0, 0, 3.1]$ → only CGPA, Backlogs, and Internships are "active"; Aptitude, Attendance, Projects were zeroed out by Lasso.

Sparsity = simpler, more interpretable model (easier to explain: "only 3 things actually matter here").

Bias vs Variance — The Deeper Explanation

Bias = error from a model that's TOO SIMPLE (it can't capture the real pattern even with unlimited data)

High bias = **underfitting**

Both training AND test error are high

Example: fitting a straight line through obviously curved data

Variance = error from a model that's TOO SENSITIVE to the specific training data it saw

High variance = **overfitting**

Training error very low, but test error high (inconsistent on new data)

Example: a wiggly curve that chases every noisy data point

Aspect	Underfitting (high bias)	Overfitting (high variance)
Model complexity	Too low	Too high
Training error	High	Very low
Testing error	High	High
Fix	Increase complexity, add features, reduce regularization	Add regularization, reduce complexity, get more data

The Bias-Variance Trade-off

Simple model → high bias, low variance

Complex model → low bias, high variance

Goal: find the sweet spot that minimizes total error on NEW data (not training data) — this is the "balanced" zone in the classic U-shaped generalization error curve.

How λ affects this trade-off:

Regularization strength	Effect
Too little (λ small)	Coefficients grow large $\rightarrow$ high variance $\rightarrow$ overfitting
Balanced	Controlled coefficients $\rightarrow$ good generalization
Too much (λ large)	Coefficients crushed too small $\rightarrow$ high bias $\rightarrow$ underfitting

As λ increases: Variance decreases, Bias increases. You're trading a bit of fit-quality on training data for much better performance on new data.

SESSION 4: FEATURE ENGINEERING (Data Preprocessing)

Why Does Raw Data Need Preprocessing?

Real-world data is messy in 4 common ways:

Problem	Example
Different ranges	CGPA is 0–10, but Attendance is 40–100
Text categories	Branch = "ECE", "CSE" (models need numbers, not text)
Missing values	CGPA = blank/NaN for some students
Extreme values (outliers)	A test score of 42 among a bunch of 90s

Preprocessing = converting messy raw data into a clean representation the algorithm can actually use.

Part A: Feature Scaling

What it is: transforming numerical values onto comparable scales WITHOUT changing their order or meaning.

Why it matters: algorithms like KNN, logistic regression (via gradient descent), SVMs, and neural networks are sensitive to the raw magnitude of numbers.

Without proper scaling:

Large-valued features can dominate distance calculations

Gradient descent converges slowly or unevenly

Regularization penalizes features unfairly (a feature with naturally bigger numbers looks "more important" even if it isn't)

With proper scaling: everything above works more smoothly and fairly.

Method 1: Standardization (StandardScaler)

$$x' = \frac{x - \mu}{\sigma}$$

μ = mean, σ = standard deviation

Result: mean becomes ≈ 0 , standard deviation becomes ≈ 1

Values can be negative, zero, or positive (NOT restricted to $[0,1]$)

Interpretation: negative = below average, 0 = exactly average, positive = above average

Worked example: Marks = [40, 60, 80], mean = 60, $\sigma \approx 16.33 \rightarrow 40$ becomes ≈ -1.225 , 60 becomes 0, 80 becomes ≈ 1.225 (Each value now tells you "how many standard deviations away from average.")

Method 2: Min-Max Scaling

$$x' = \frac{x - x_{min}}{x_{max} - x_{min}}$$

Forces everything into a fixed range, usually $[0, 1]$

Minimum value $\rightarrow 0$, Maximum value $\rightarrow 1$

Preserves relative order but is very sensitive to extreme values

Worked example: Marks = [40, 60, 80], min=40, max=80 $\rightarrow [0, 0.5, 1]$

Method 3: RobustScaler

$$x' = \frac{x - \text{Median}}{IQR}$$

Uses median and interquartile range ($IQR = Q3 - Q1$) instead of mean/std

Why "robust"? Median and IQR aren't dragged around by extreme outliers the way mean and standard deviation are

Worked example: [30, 35, 40, 45, 100]. Median=40, IQR=10 $\rightarrow [-0.25, -0.125, 0, 0.125, 1.5]$ Notice how the outlier (100) doesn't distort the other values as much as it would with standardization.

Which Scaler to Choose?

Situation	Use
General-purpose, gradient-based models	StandardScaler
Need a strict bounded range (e.g., neural net inputs)	MinMaxScaler
Data has strong outliers	RobustScaler

Part B: Outlier Detection

An **outlier** is a data point that differs substantially from the rest. It might be a data-entry error, a sensor glitch, OR a genuine rare event — you must investigate before deleting it, never delete automatically.

Method 1: Standard Deviation Rule

$$\text{Outlier if } x < \mu - k\sigma \text{ or } x > \mu + k\sigma$$

Typically $k = 2$ or 3 .

Worked example: $[1, 2, 2, 3, 1, 3, 10]$ $\mu \approx 3.143$, $\sigma \approx 2.917$ Lower limit = $3.143 - 2(2.917) \approx -2.691$ Upper limit = $3.143 + 2(2.917) \approx 8.977$ Since $10 > 8.977 \rightarrow$ **10 is an outlier**

(Limitation: mean/std are themselves distorted by outliers, so this can be unreliable for very skewed data.)

Method 2: Z-Score

$$z = \frac{x - \mu}{\sigma}$$

This is literally the same formula as standardization! The Z-score tells you how many standard deviations a point is from the mean.

Rule of thumb:

$|z| > 2 \rightarrow$ mild outlier

$|z| > 3 \rightarrow$ extreme outlier

Worked example: same dataset, for $x=10$: $z \approx 2.35 \rightarrow |z| > 2 \rightarrow$ **mild outlier**. For $x=1$ or $x=3$, $|z| < 2 \rightarrow$ not outliers.

Method 3: IQR Method (most robust to skew)

$$IQR = Q3 - Q1$$

$$\text{Lower bound} = Q1 - 1.5 \times IQR \quad \text{Upper bound} = Q3 + 1.5 \times IQR$$

Q1 = 25th percentile (middle of the lower half)

Q3 = 75th percentile (middle of the upper half)

Worked example: [1,1,2,2,3,3,10] (sorted) Q2 (median) = 2, Q1 = 1, Q3 = 3 IQR = 3-1 = 2 Lower bound = 1 - 1.5(2) = -2 Upper bound = 3 + 1.5(2) = 6 Since 10 > 6 → **10 is an outlier**

Part C: Encoding Categorical Data

Models need numbers, not text like "ECE" or "Low/Medium/High." Different situations call for different encodings.

One-Hot Encoding

Creates a separate binary (0/1) column for each category. Use when categories have **no natural order**.

Branch **Branch_ECE** **Branch_CSE** **Branch_ME**

ECE 1 0 0

CSE 0 1 0

ME 0 0 1

Limitation: if a feature has 50 categories, you get 50 new columns — can get unwieldy.

Label Encoding

Just assigns each category a number: ECE→0, CSE→1, ME→2. **Caution:** these numbers don't imply any real order or ranking — the model might mistakenly think ME (2) is "more" than ECE (0).

Ordinal Encoding

Same idea as label encoding, but used **ONLY** when there genuinely IS a meaningful order: Low→0, Medium→1, High→2. Here the numeric order actually reflects reality.

Target Encoding

Replace a category with the average target value for that category (e.g., Delhi → average salary of Delhi employees = 55).

Big caution: Must be calculated using ONLY the training data. If you accidentally use test-set outcomes to compute these averages, you leak information from the test set into training — called **data leakage** — and your model will look better than it actually is.

Part D: Missing-Value Imputation

Real datasets often have gaps. You can't just delete every row with a missing value (you'd lose too much data) — instead, you fill in ("impute") a reasonable estimate.

Method	How it works	Best for
Mean imputation	Fill with the column average	Numerical data, no strong skew/outliers
Median imputation	Fill with the middle value	Numerical data WITH outliers/skew
Mode imputation	Fill with the most frequent value	Categorical variables

Critical rule (same data leakage issue as above): imputation values (the mean/median/mode used) must be calculated from the TRAINING set only, then applied to validation/test/new data. If you compute these using the full dataset (including test data), information leaks and your evaluation becomes unreliable.

HOW THE FOUR SESSIONS CONNECT

Think of building a real ML model as a pipeline:

Feature Engineering (Session 4) — clean your raw data first: scale numbers, handle outliers, encode categories, fill missing values

Choose a model type:

Predicting a number? → **Linear Regression (Session 1)**

Predicting a category? → **Logistic Regression (Session 2)**

Train it using MSE (linear) or Cross-Entropy (logistic) + Gradient Descent or the Normal Equation

Prevent overfitting using **Regularization (Session 3)** — Ridge/Lasso/Elastic Net — so the model generalizes to new data instead of just memorizing training data

Evaluate using MSE/RMSE/R² (regression) or accuracy/cross-entropy (classification), always checking performance on data the model hasn't seen (test set) to catch overfitting

This is essentially the entire classical ML workflow in miniature — everything you'll do with more advanced models later builds on exactly these ideas.

QUICK SELF-TEST (from the slides — try answering these yourself!)

Which method directly computes regression coefficients using matrix algebra? (*Normal Equation*)

What does the slope represent in simple linear regression? (*How much y changes per unit of x*)

Why can't linear regression output directly be used as a probability? (*Unbounded range, doesn't fit within 0–1*)

What is $\sigma(0)$? (*0.5*)

Which loss function is used for binary classification? (*Binary cross-entropy / log loss*)

Which regularization method can set coefficients to exactly zero? (*Lasso / L1*)

Is underfitting linked to high bias or high variance? (*High bias*)

Which scaling method is least affected by outliers? (*RobustScaler*)

Why must imputation/encoding values be calculated from training data only? (*To avoid data leakage — accidentally "peeking" at test data during training*)